Table of Contents

Running Native Code from a Flutter App **1**
Flutter's Platform Channel for Communicating with Native Code 1
Invoking Native Android Code with Flutter 2
Checking the Battery Level with Native Android API from Flutter **6**
FutureBuilder Widgets **15**
Using the Camera Function **19**
Enabling the Camera for Android 19
Obtaining the Dependencies to Support the Camera App 20
Saving to the Photo in Android 25

Running Native Code from a Flutter App

While Flutter provides a robust framework for building cross-platform applications, certain situations may require writing native code. The need to write native code is a normal scenario for all cross mobile platform frameworks including React Native, Ionic and Xamarin. In other words, regardless of the cross mobile framework you choose, you may occasionally need to write some code either in Kotlin or Java code for Android and either in SwiftUI or Objective-C code for iOS.

Below are some scenarios where native code is necessary:

- **Native APIs and SDKs** may be required when Flutter plugins are either unavailable or unable to provide deep enough integration.
- **Performance Optimization** is needed for performance-critical applications (like game engines or real-time processing).
- **Low-level System Features** are not exposed through Flutter's framework. Examples could include, system-level services like background processing, hardware acceleration, or specific security features, such as using the Android Keystore or iOS Keychain.
- **Custom UI Components** are needed when these UI components that are not feasible with Flutter's widget system. For example, this could be necessary when creating a custom camera view or a proprietary UI element that requires direct access to the native rendering system.
- **Achieving Security and Compliance** may sometimes be better handled with native code.

Flutter's Platform Channel for Communicating with Native Code

Flutter uses a flexible message system called a **Platform Channel** that allows you to call platform-specific APIs in a native language that works directly with those APIs including:

- Kotlin or Java on Android
- Swift or Objective-C on iOS
- C++ on Windows
- Objective-C on macOS
- C on Linux

To enable communication between native code and Flutter:

1. **The Flutter portion of the app** sends messages to its host, the non-Dart portion of the app, over a platform channel.

2. **The host** listens on the platform channel, and receives the message. It then calls into any number of platform-specific API using the native programming language and sends a response back to the Flutter portion of the app (see Figure 1).

Figure 1: Message flow across platform channel.

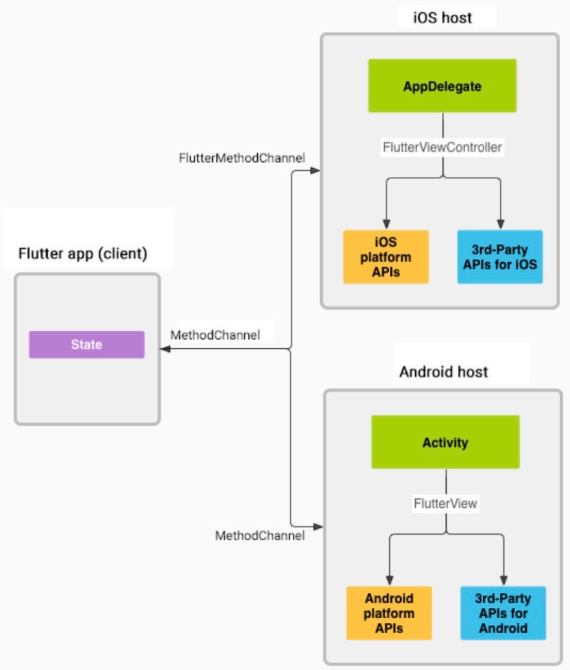


Image from https://docs.flutter.dev/platform-integration/platform-channels

Invoking Native Android Code with Flutter

Example 1: Invoking Native Android Code

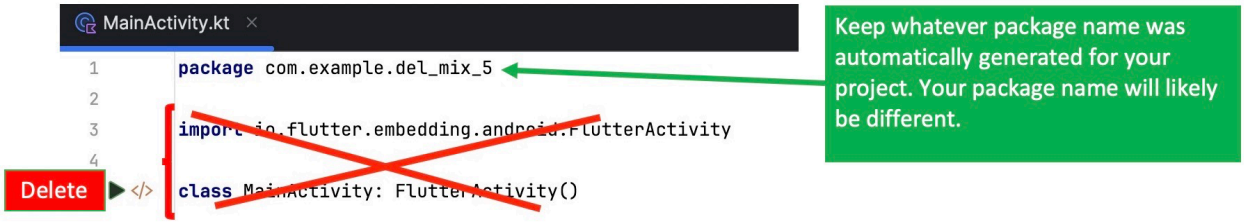
This example demonstrates how to invoke native Kotlin code from Flutter. The top button triggers a message from Flutter across the platform channel. The native Kotlin code responds by sending a response back to Flutter when the button is pressed.

Figure 2: Invoking native code with Flutter.

To make this work, start with a project that already successfully runs a Flutter app in the Android emulator.

These next sevem steps are very important so please pay close attention:

1. Create a Flutter app.
2. After ensuring your Flutter app runs using an emulator (or device), stop the app from running. You can leave your emulator running though. Then, choose **File | Close Project**. When that step is completed, no Android projects will be open on your computer but Android Studio will still be running so another project can be opened.
3. Next choose **File | Open** and navigate to the **android** sub folder of the same project and open that folder in Android Studio.
4. In **app\src\main\Kotlin\com.yourpackage\MainActivity.kt** delete all contents of the file except the top line which lists that package that was automatically generated for your project.



5. Add the following code <u>after the package reference that was automatically generated for your project</u>.

MainActivity.kt

```kotlin
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel
import android.util.Log

class MainActivity: FlutterActivity() {
    private val channelName = "samples.calling_native_code"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)

        val method = MethodChannel(flutterEngine.dartExecutor.binaryMessenger,
            channelName)
        method.setMethodCallHandler { call, result ->
            if (call.method == "getGreeting") {
                val fname = call.argument<String>("fname")
                val greetingMessage = "Hello, $fname from native code."
                Log.d("***", greetingMessage)
                result.success(greetingMessage) // Send the greeting back to Flutter
            }
        }
    }
}
```

6. After editing the Kotlin file choose **File | Close Project**.
7. Next choose **File | Open** and navigate to the root of the original Flutter project to reopen it. Then add the following code into main.dart.

main.dart

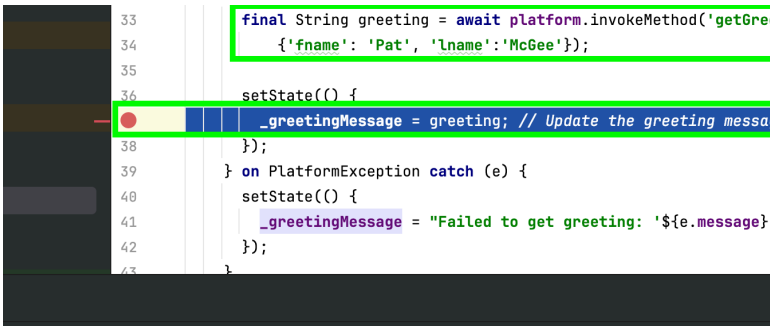
```dart
import 'package:flutter/material.dart';
import 'dart:async';
import 'package:flutter/services.dart';
import 'dart:developer'; // Enables logging for emulator and physical devices.

void main() => runApp(const ShowPlatformChannelApp());

class ShowPlatformChannelApp extends StatelessWidget {
  const ShowPlatformChannelApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      restorationScopeId: 'app',
      home: PlatformChannelApp(),
    );
  }
}

class PlatformChannelApp extends StatefulWidget {
  const PlatformChannelApp({super.key});

  @override
  _PlatformChannelState createState() => _PlatformChannelState();
}

class _PlatformChannelState extends State<PlatformChannelApp> {
  var platform = const MethodChannel('samples.calling_native_code');

  String _greetingMessage = "";
  Future<void> _getGreetingFromNativeCode() async {
    try {
      final String greeting = await platform.invokeMethod('getGreeting',
          {'fname': 'Pat'});

      setState(() {
        _greetingMessage = greeting; // Update the greeting message
      });
    } on PlatformException catch (e) {
      setState(() {
        _greetingMessage = "Failed to get greeting: '${e.message}'.";
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Material(
```

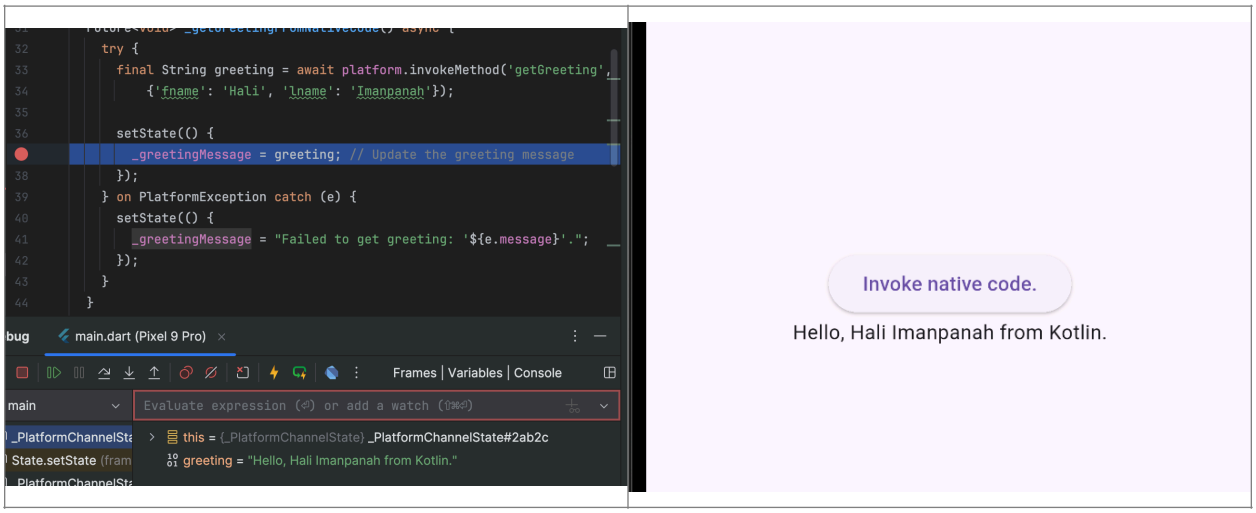
Select an Android emulator and run the project. The project should allow you to click the button to send a message to native Kotlin code which will respond back to Flutter with a message.

Exercise 1 (4 marks)

Modify the code in Example 1 for Android to send your last name in addition to your first name to the native code. The greeting from the native code should respond by spelling out <u>your full name</u>. Create a screenshot while halted at the breakpoint as shown in Figure 3 but make sure it shows your name. Note that the three highlighted pieces of information need to be in the screenshot and most importantly the program must be halted at the breakpoint to get the mark.

Figure 3: Receiving first and last name in greeting back from native code.

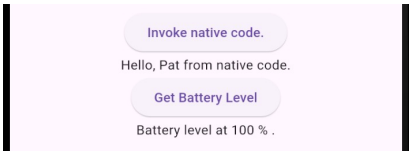




Checking the Battery Level with Native Android API from Flutter

Example 2: Checking the Battery Level in Android

This example demonstrates how Flutter can invoke the native Android BatteryManager API to report the battery level.



Starting with Example 1, add the following imports to MainActivity.kt.

android\app\src\main\kotlin\com\example\<projectname>\MainActivity.kt

```
import androidx.annotation.NonNull
import android.content.Context
import android.content.ContextWrapper
import android.content.Intent
import android.content.IntentFilter
import android.os.BatteryManager
import android.os.Build.VERSION
import android.os.Build.VERSION_CODES
```

Then add this function to the MainActivity class read the battery level. For a better editing experience close the Flutter project by navigating to File | Close Project in Android Studio. Then, from Android Studio choose Open and navigate to the android folder before opening it.

MainActivity.kt

```
private fun getBatteryLevel(): Int {
    val batteryLevel: Int
    if (VERSION.SDK_INT >= VERSION_CODES.LOLLIPOP) {
        val batteryManager = getSystemService(Context.BATTERY_SERVICE)
            as BatteryManager
        batteryLevel = batteryManager.getIntProperty(
                        BatteryManager.BATTERY_PROPERTY_CAPACITY)
    } else {
        val intent = ContextWrapper(applicationContext).registerReceiver(null,
            IntentFilter(Intent.ACTION_BATTERY_CHANGED))
        batteryLevel = intent!!.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) * 100 /
intent.getIntExtra(BatteryManager.EXTRA_SCALE, -1)
    }
    return batteryLevel
}
```

Next, add this conditional block to execute a battery check if requested through the platform channel.

MainActivity.kt

```
else if (call.method == "getBatteryLevel") {
    val batteryLevel = getBatteryLevel()

    if (batteryLevel != -1) {
        result.success(batteryLevel)
    } else {
        result.error("UNAVAILABLE", "Battery level not available.", null)
    }
} else {
    result.notImplemented()
}
```

Then, inside **lib\main.dart** include this code within the state object to make the request for the battery level through the platform channel.

main.dart

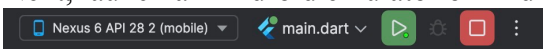
```
String _batteryLevel = 'Unknown battery level.';

Future<void> _getBatteryLevel() async {
    String batteryLevel;
    try {
      final result = await platform.invokeMethod<int>('getBatteryLevel');
      batteryLevel = 'Battery level at $result % .';
    } on PlatformException catch (e) {
      batteryLevel = "Failed to get battery level: '${e.message}'.";
    }
    setState(() {
      _batteryLevel = batteryLevel;
    });
}
```

Finally, add these widgets inside the build function of the state object to show the battery level once it has been retrieved back from the native code.

main.dart

```
ElevatedButton(
    onPressed: _getBatteryLevel,
    child: const Text('Get Battery Level'),
),
Text(_batteryLevel),
```

Next, launch an Android emulator or Android device to test the code to be sure it works.



Exercise 2 (4 marks)

Modify the code either for Example 2: Checking the Battery Level in Android. Change the code so a different channel is used than the channel which the greeting is on. Show your modified dart code here:

```
package com.example.lab_5

import 'package:flutter/material.dart';
import 'dart:async';
import 'package:flutter/services.dart';
import 'dart:developer';

void main() => runApp(const ShowPlatformChannelApp());

class ShowPlatformChannelApp extends StatelessWidget {
  const ShowPlatformChannelApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      restorationScopeId: 'app',
      home: PlatformChannelApp(),
    );
  }
}

class PlatformChannelApp extends StatefulWidget {
  const PlatformChannelApp({super.key});

  @override
```

```
  _PlatformChannelState createState() => _PlatformChannelState();
}

class _PlatformChannelState extends State<PlatformChannelApp> {

  var greetingChannel = const MethodChannel('samples.flutter.dev/greeting');
  var batteryChannel = const MethodChannel('samples.flutter.dev/battery');

  String _greetingMessage = "";
  String _batteryLevel = "";
  Future<void> _getGreetingFromNativeCode() async {
    try {
      final String greeting = await
greetingChannel.invokeMethod('getGreeting',
          {'fname': 'Hali', 'lname': 'Imanpanah'});

      setState(() {
        _greetingMessage = greeting; // Update the greeting message
      });
    } on PlatformException catch (e) {
      setState(() {
        _greetingMessage = "Failed to get greeting: '${e.message}'.";
      });
    }
  }
  Future<void> _getBatteryLevel() async {
    try {
      final int result = await
batteryChannel.invokeMethod('getBatteryLevel');
      setState(() {
```

```
        _batteryLevel = 'Battery level: $result%';
      });
    } on PlatformException catch (e) {
      setState(() {
        _batteryLevel = "Failed to get battery level: '${e.message}'.";
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Material(
      child: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [

            ElevatedButton(
              onPressed: _getGreetingFromNativeCode,
              child: const Text('Invoke native code.'),

            ),

            Text(_greetingMessage),
            ElevatedButton(
              onPressed: _getBatteryLevel,
              child: const Text('Get Battery Level'),
            ),
            Text(_batteryLevel),
```

```
          ],
        ),
      ),
    );
  }
}
```

Show your modified native Android code here:

```
package com.example.lab_5

import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel
import android.util.Log
import android.content.Context
import android.content.ContextWrapper
import android.content.Intent
import android.content.IntentFilter
import android.os.BatteryManager
import android.os.Build.VERSION
import android.os.Build.VERSION_CODES


class MainActivity: FlutterActivity() {
    private val greetingChannelName = "samples.flutter.dev/greeting"
    private val batteryChannelName  = "samples.flutter.dev/battery"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
```

```
        super.configureFlutterEngine(flutterEngine)

        val greetingMethod=
MethodChannel(flutterEngine.dartExecutor.binaryMessenger,
            greetingChannelName)

        greetingMethod.setMethodCallHandler { call, result ->

            if (call.method == "getGreeting") {
                val fname = call.argument<String>("fname")
                val lname = call.argument<String>("lname")
                val greetingMessage = "Hello, $fname $lname from native
code."
                Log.d("***", greetingMessage)
                result.success(greetingMessage)

            } else {
                result.notImplemented()
            }
        }


    // Battery channel
    MethodChannel(flutterEngine.dartExecutor.binaryMessenger,
batteryChannelName)
    .setMethodCallHandler { call, result ->
        if (call.method == "getBatteryLevel") {
            val batteryLevel = getBatteryLevel()
            if (batteryLevel != -1) {
                result.success(batteryLevel)
```

```
            } else {
                result.error("UNAVAILABLE", "Battery level not available.",
null)
            }
        } else {
            result.notImplemented()
        }
    }
}
```

```
    private fun getBatteryLevel(): Int {
        val batteryLevel: Int
        if (VERSION.SDK_INT >= VERSION_CODES.LOLLIPOP) {
            val batteryManager = getSystemService(Context.BATTERY_SERVICE)
                    as BatteryManager
            batteryLevel = batteryManager.getIntProperty(
                BatteryManager.BATTERY_PROPERTY_CAPACITY)
        } else {
            val intent = ContextWrapper(applicationContext).registerReceiver(
                null,
                IntentFilter(Intent.ACTION_BATTERY_CHANGED)
                )

            batteryLevel =
                intent!!.getIntExtra(BatteryManager.EXTRA_LEVEL, -1) * 100 /
                        intent.getIntExtra(BatteryManager.EXTRA_SCALE, -1)
        }
        return batteryLevel
```

```
    }
}
```

FutureBuilder Widgets

To prepare for using the camera, this section will discuss how to use FutureBuilder. FutureBuilder shows one of multiple widgets according to the state of an asynchronous operation.

The FutureBuilder has two attributes:

- The **future** attribute invokes an asynchronous operation to get a resource.
- The **builder** attribute defines multiple widgets. Only one displays depending on the state of the asynchronous operation.

Example 3; Future Builder Introduction

This example demonstrates how a FutureBuilder widget displays different UI depending on the resource status.

Figure 4: Showing a spinner before the resource is available (left). Showing the data after it has been received (right).

future_builder.dart

```dart
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';

// A function to build the FutureBuilder widget.
Widget buildFutureData(Future<String> Function() futureFunc) {
  return FutureBuilder<String>(
    // Resource to be fetched asynchronously.
    future: futureFunc(),

    // Defines what to display based on future's state
    builder: (BuildContext context, AsyncSnapshot<String> snapshot) {
      // Check the connection state
      if (snapshot.connectionState == ConnectionState.waiting) {
        // While the future is resolving, show a loading indicator
        return CircularProgressIndicator();
      } else if (snapshot.hasError) {
        // If the future encountered an error, display the error
        return Text('Error: ${snapshot.error}');
      } else {
        // When the future completes successfully, display the result
        return Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              'Fetched data: ${snapshot.data}',
              style: TextStyle(fontSize: 24),
              textAlign: TextAlign.center, // Center text
            ),
          ],
        );
      }
    },
  );
}
```

main.dart

```dart
import 'dart:async';
import 'package:flutter/material.dart';
import 'future_builder.dart';

void main() {
  runApp(MyApp());
}

/// The main app widget.
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'FutureBuilder Example',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: FutureBuilderExample(),
    );
  }
}

/// A widget that demonstrates the use of FutureBuilder.
class FutureBuilderExample extends StatefulWidget {
  @override
  _FutureBuilderExampleState createState() => _FutureBuilderExampleState();
}

class _FutureBuilderExampleState extends State<FutureBuilderExample> {
  // Simulating a network call that returns a Future.
  Future<String> fetchData() async {
    // Simulate a network delay
    await Future.delayed(Duration(seconds: 2));
    return 'Hello, FutureBuilder!';
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('FutureBuilder'),
      ),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(16.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              // Call the new FutureBuilder function with fetchData
              buildFutureData(fetchData),
```

Exercise 3 (1 mark)

What snapshot property is used to check if the data is ready?

```
snapshot.connectionState
```

Exercise 4 (2 marks)

Examine the **future_builder.dart** code and explain what the snapshot does.

snapshot is a box which contains the data and current state of loading.It lets the program know: if the data is still loading, if an error is happened (there is an error) or if the data is ready to use

Exercise 5 (2 marks)

Explain in your own words what each of the three conditional blocks in **future_builder.dart** do in Example 3.

```
snapshot.connectionState == ConnectionState.waiting
```

First block runs when the program is still waiting for the data, then it shows a loading spinner to tell us (user) that something is happening.

```
snapshot.hasError
```

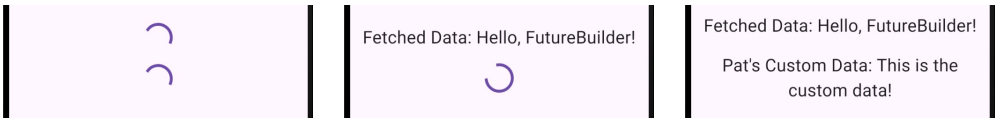
Second block runs if an error happens while getting the data, then it shows an error message to let the user know something went wrong.

```
else
```

Last block runs when the data has been received successfully, so it displays the fetched data on the screen.

Exercise 6 (4 marks)

Modify Example 3 by adding another FutureBuilder widget to your UI. It should wait longer than the first FutureBuilder widget. The output will look like the following at the different stages but it will show your name.



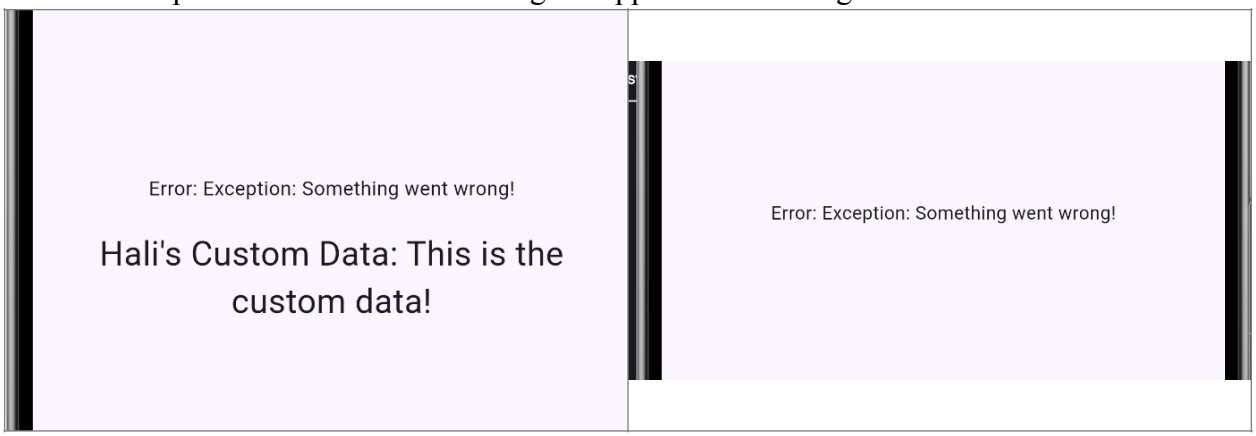
Please show three separate screenshots that show your working app with your name in it:

Exercise 7 (1 mark)

Modify the fetchData() function so it throws an error:

```
Future<String> fetchData() async {
  await Future.delayed(Duration(seconds: 2));
  throw Exception('Something went wrong!');
}
```

Show the output that occurs when running the app after this change.



Using the Camera Function

Accessing the camera for taking pictures and recording videos is relatively easy to implement by using Flutter dependencies which work for both Android and iOS. The **camera** dependency provides tools to work with the cameras on the device. The **path_provider** dependency finds the correct path to store images. The **path** dependency creates paths that work on any platform.

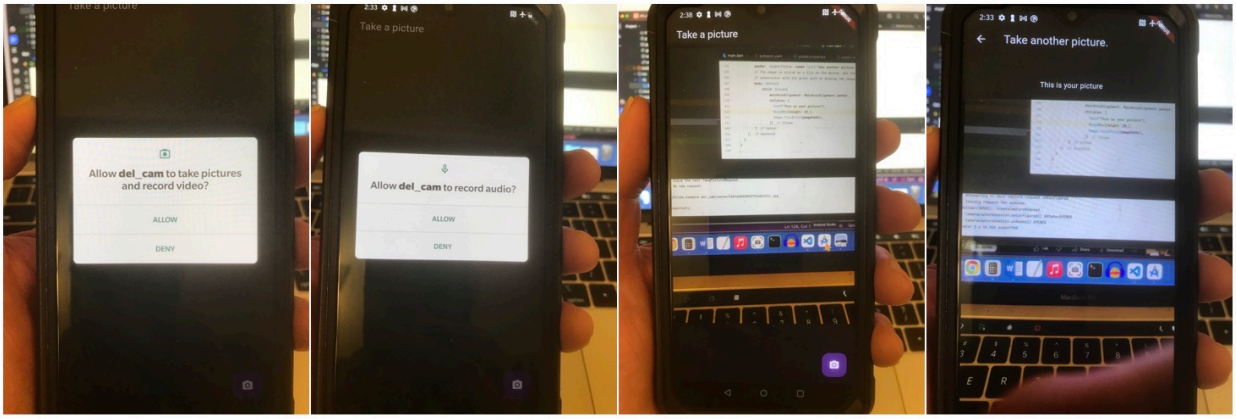
Enabling the Camera for Android

Example 4: Enabling Camera for Android

This example sets up the camera for Android specifically. This example works best in an actual Android phone. However, the emulator allows you to simulate taking a picture if you do not have an Android phone that is not connected to your computer.

When the app first launches, it will ask for permission to use the microphone and camera. The dependency used for the camera also requires access to the microphone so the prompt is necessary. However, the user will only be prompted once.

After granting permission, you then can focus and take the shot as shown in the second image from the right. Once the picture is taken, it is displayed and then you can navigate to the original screen and do it again.

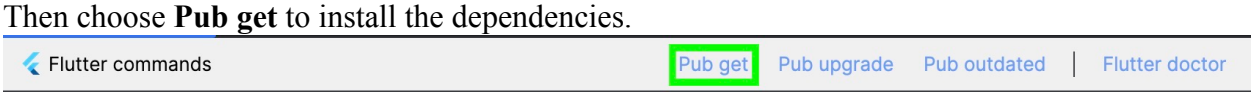


Obtaining the Dependencies to Support the Camera App

To complete this application, add three dependencies to your app. To get these dependencies, inside **pubspec.yaml** enter the highlighted dependencies.

```
dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  camera: ^0.11.2
  path_provider: ^2.1.5
  path: ^1.9.1
```

Then choose **Pub get** to install the dependencies.



Note: Instead of editing the pubspec.yaml file you can run the following command in the Android Studio terminal. Running this command will obtain current versions:

```
flutter pub add camera path_provider path
```

Tip: This demo will work if you test it on your Android device. Since the emulator does not have a real camera it will show a screen that looks like something from Minecraft for the simulation. If you get the Minecraft screen on your emulator then it is working properly.

WidgetsFlutterBinding.ensureInitialized() ensures that the Flutter framework is initialized before accessing platform channels. This is necessary before calling any asynchronous method that interacts with plugins.

availableCameras() asynchronously fetches a list of available cameras on the device.

firstCamera gets the first camera from the list.

The **FutureBuilder** widget waits for the camera controller to initialize. If initialized, it shows the camera preview; otherwise, it shows a loading indicator.

display_picture_screen.dart

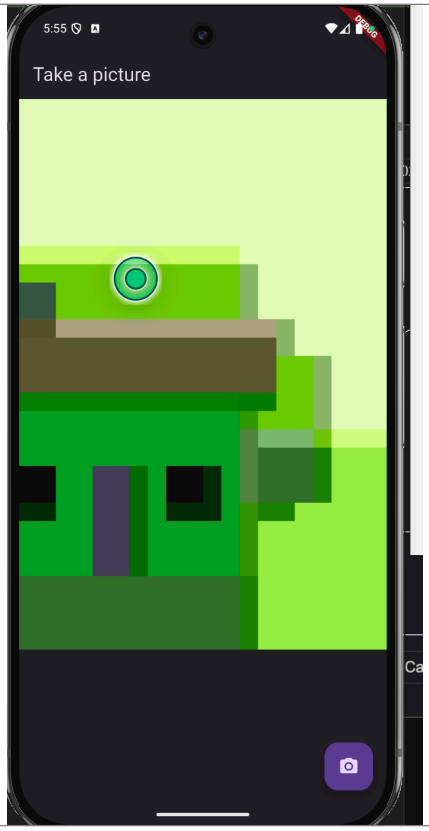
```
// A widget that displays the picture taken by the user.
import 'dart:io';

import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';

class DisplayPictureScreen extends StatelessWidget {
  final String imagePath;

  const DisplayPictureScreen({super.key, required this.imagePath});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: const Text('Take another picture.')),
        // The image is stored as a file on the device. Use the `Image.file`
        // constructor with the given path to display the image.
        body: Center(
            child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Text("This is your picture"),
                  SizedBox(height: 20,),
                  Image.file(File(imagePath)),
                ])
        )
    );
  }
}
```

main.dart

```dart
import 'dart:async';
import 'dart:developer';
import 'package:camera/camera.dart';
import 'package:flutter/material.dart';
import 'display_picture_screen.dart';

Future<void> main() async {
  // Ensure that plugin services are initialized so that `availableCameras()`
  // can be called before `runApp()`
  WidgetsFlutterBinding.ensureInitialized();

  // Get all cameras and choose the first one.
  final cameras     = await availableCameras();
  final firstCamera = cameras.first;

  runApp(
    MaterialApp(
      theme: ThemeData.dark(),
      home: TakePictureScreen(
        // Pass the appropriate camera to the TakePictureScreen widget.
        camera: firstCamera,
      ),
    ),
  );
}

// A screen that allows users to take a picture using a given camera.
class TakePictureScreen extends StatefulWidget {
  const TakePictureScreen({super.key, required this.camera});

  final CameraDescription camera;

  @override
  TakePictureScreenState createState() => TakePictureScreenState();
}

class TakePictureScreenState extends State<TakePictureScreen> {
  late CameraController _controller;
  late Future<void> _initializeControllerFuture;

  @override
  void initState() {
    super.initState();
    // Initialize the camera controller.
    _controller = CameraController(widget.camera, ResolutionPreset.medium);
    _initializeControllerFuture = _controller.initialize();
  }
```

Exercise 8 (2 marks)

Take a screenshot of the camera output in your emulator and show it here. Note that this demo will work if you test it on your Android device. Since the emulator does not have a real camera it will show a screen that looks like something from Minecraft for the simulation. If you get the Minecraft screen on your emulator then it is working properly.



Exercise 9 (1 mark)

What is the role of the following code?

```
WidgetsFlutterBinding.ensureInitialized();
```

To makes sure Flutter is fully set up before using plugins/ async functions (ex: camera).

Exercise 10 (1 mark)

Which method is used to retrieve the list of available cameras on the device?

```
availableCameras()
```

Exercise 11 (1 mark)

What function does the future builder wait for? What does it do once the resource is available?

`_initializeControllerFuture`

It waits for the camera to finish getting ready.
When the camera is ready, it shows the camera view on the screen instead of the loading circle.

Exercise 12 (1 mark)

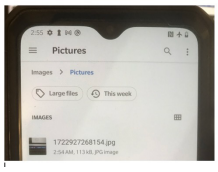
What does the future show before the resource is available?

A loading spinner

Saving to the Photo in Android

Example 5: Saving to Gallery in Android

This example demonstrates how to save the image after taking a picture on an Android device. When the picture has been taken you will be able to view it later by navigating to your files | images area in Android.



WARNING: This example only works on a real camera. It does not work on the emulator.

Starting with Example 4, add the highlighted references below and choose **Pub Get** to download the packages which enable access to the image file area.

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  camera: ^0.11.2
  path_provider: ^2.1.5
  path: ^1.9.1
  permission_handler: ^12.0.1
  image_gallery_saver_plus: ^4.0.1
```

Next, add the following permissions in AndroidManifest.xml.

android/app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.INTERNET"/>
    <application
```

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
```

Next, add the import

```
import 'package:permission_handler/permission_handler.dart';
```

Then, add the following code to the TakePictureScreenState class.

```
Future<void> _saveImage(image) async {
  if (image == null) return;

  // Request storage permission
  if (await Permission.storage.request().isGranted) {
    print("Saving to image with Pat's app.");

    final result = await ImageGallerySaverPlus.saveFile(image!.path);
    print("*** Pat's app saved the image.");

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Image Saved: $result')),
    );
  } else {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text('Storage Permission Denied')),
    );
  }
}
```

Note: If the Permission and ImageGallerySaverPlus classes are not recognized, be sure to choose Pub Get. You might also try restarting Android Studio. Sometimes the restart causes the dependencies to load.

Call _saveImage() before navigating to the screen after the picture is taken.

```
await _saveImage(image);
```

To check, if you have an Android device, run the app and take a picture. The image should appear in the images area where your files are saved.